Tutorial_5_Voice_to_Code

Tutorial 5: Voice-to-Code Workflow

Duration: 15 minutes **Level**: Beginner

Overview

Code hands-free using voice input: - Setup Whisper transcription - Voice commands - Dictate code - Voice-controlled refactoring

Step 1: Setup Voice Input

```
# Requires OpenAI API key for Whisper
export OPENAI_API_KEY=sk-...

helixcode voice init

# Testing microphone... ✓
# Whisper API configured ✓
```

Step 2: Voice Commands

```
helixcode voice

# Say: "Create a function to validate email addresses"
# HelixCode transcribes and generates code
```

Generated:

```
func ValidateEmail(email string) bool {
    regex := regexp.MustCompile(`^[a-z0-9._%+\-]+@[a-z0-9.\-]+\.
        [a-z]{2,}$`)
    return regex.MatchString(email)
}
```

Step 3: Voice-Dictated Refactoring

```
# Say: "Refactor the authentication module to use dependency
        injection
#       Extract an interface called AuthProvider
#       Implement JWT provider
#       Add unit tests"

# HelixCode transcribes and executes plan
```

Step 4: Code Review by Voice

```
helixcode voice review

# Say: "Review the user service for security issues"

# HelixCode analyzes and responds verbally
# TTS: "I found 3 potential issues:
#       1. SQL injection in GetUserByEmail
#       2. Missing input validation in CreateUser
#       3. Plain text password in logs"
```

Step 5: Hands-Free Development

```
# Start voice session
helixcode voice session

# Voice commands:
# "Create REST API endpoint for user registration"
# "Add validation"
# "Write unit tests"
# "Commit with message: Add user registration"
# "Push to GitHub"
```

Configuration

```
tools:
  voice:
    enabled: true
    model: "whisper-1"
    language: "en"
    sample_rate: 16000
    silence_timeout: 2s

    # Wake word (optional)
    wake_word: "helix"

    # Text-to-speech for responses
    tts:
      enabled: true
      voice: "nova"
```

Use Cases

- **Accessibility**: Developers with physical limitations
- **Multitasking**: Code while cooking, walking, etc.
- **Brainstorming**: Verbal architecture discussions
- **Pair Programming**: Voice-driven collaborative coding

Results

- **Hands-Free**: Code without keyboard
- **Fast Input**: Speak faster than you type
- **Natural**: Conversational programming

Continue to Tutorial 6: Multi-File Atomic Edits

Sources verified

Sources verified 2026-05-29: https://developers.openai.com/docs/guides/speech-to-text + https://developers.openai.com/docs/guides/text-to-speech (OpenAI Audio API, via Context7 mirror of developers.openai.com — platform.openai.com itself returns HTTP 403 to automated fetch). CONFIRMED CURRENT: `whisper-1` is still a valid transcription model and `nova` is still a valid TTS voice as of this date. ENHANCEMENT NOTE — OpenAI now also offers higher-quality transcription models `gpt-4o-transcribe`, `gpt-4o-mini-transcribe`, and `gpt-4o-transcribe-diarize`; current TTS uses the `gpt-4o-mini-tts` model (the older `tts-1`/`tts-1-hd` still work) with the same `nova` voice. Nothing in this tutorial is stale, but operators wanting best quality may prefer the gpt-4o transcription models. NEGATIVE FINDING — OpenAI's `platform.openai.com` docs are auth-gated (HTTP 403) to direct fetch; verification was performed against the public `developers.openai.com` documentation source.